

```

from time import perf_counter

GOAL = "123456780"    # 0 represents the blank space

# Possible moves for each blank position
MOVES = {
    0: [1, 3],
    1: [0, 2, 4],
    2: [1, 5],
    3: [0, 4, 6],
    4: [1, 3, 5, 7],
    5: [2, 4, 8],
    6: [3, 7],
    7: [4, 6, 8],
    8: [5, 7]
}

def get_neighbors(state):
    """Generate all states obtained by moving the blank."""
    zero = state.index("0")

    for move in MOVES[zero]:
        state_list = list(state)
        state_list[zero], state_list[move] = state_list[move],
state_list[zero]
        yield "".join(state_list)

def dls(state, goal, limit, path, visited):
    """Depth-Limited DFS."""
    if state == goal:
        return path

    if limit == 0:
        return None

    for next_state in get_neighbors(state):
        if next_state not in visited:
            visited.add(next_state)

```

```

        result = dls(
            next_state,
            goal,
            limit - 1,
            path + [next_state],
            visited
        )

        if result is not None:
            return result

    visited.remove(next_state)

return None

def iddfs(start, goal, max_limit):
    """Iterative Deepening DFS."""
    for depth in range(max_limit + 1):
        print("Searching at depth:", depth)

        visited = {start}
        result = dls(start, goal, depth, [start], visited)

        if result is not None:
            return result

    return None

def print_puzzle(state):
    for i in range(0, 9, 3):
        print(state[i:i + 3])
    print()

# ----- MAIN -----

start = input("Enter initial state (0 for blank): ")

```

```

limit = int(input("Enter maximum depth limit: "))

if len(start) != 9 or set(start) != set("012345678"):
    print("Invalid puzzle state.")
else:
    start_time = perf_counter()

    solution = iddfs(start, GOAL, limit)

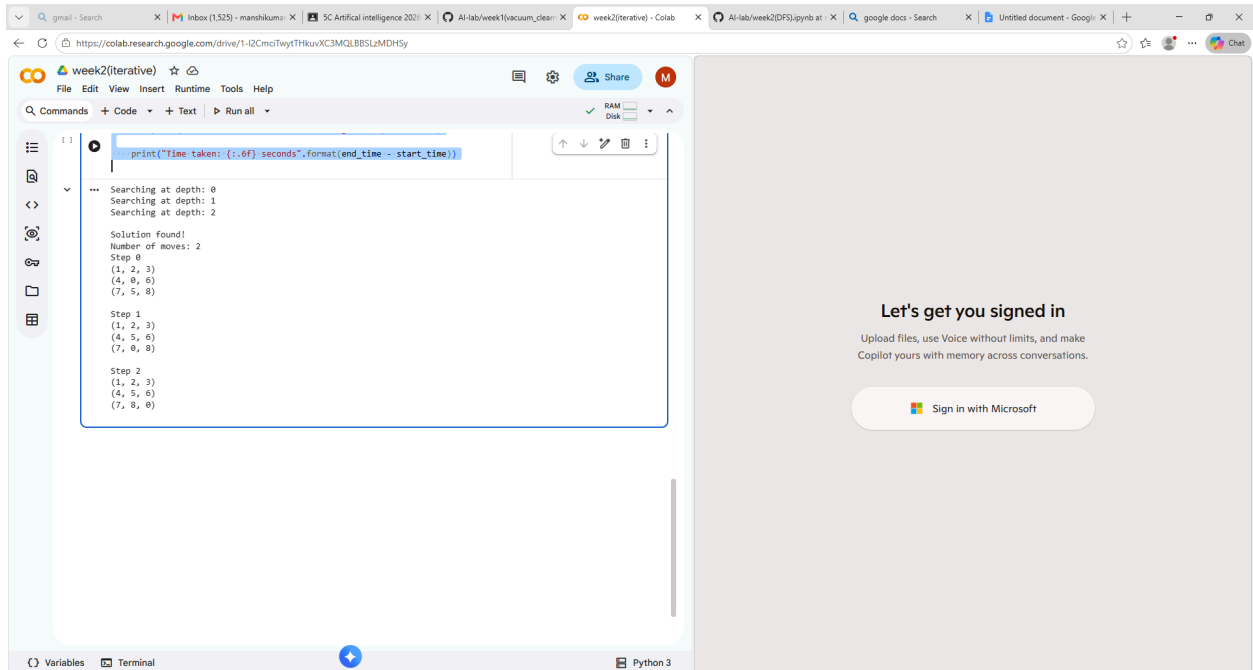
    end_time = perf_counter()

    if solution:
        print("\nSolution found!")
        print("Number of moves:", len(solution) - 1)

        print("\nPath:")
        for state in solution:
            print_puzzle(state)
    else:
        print("\nNo solution found within the given depth limit.")

print("Time taken: {:.6f} seconds".format(end_time - start_time))

```



The screenshot shows a Google Colab notebook titled "week2(Iterative)". The code is executed, and the output is displayed in the left pane. The output shows the search depth, the solution found, the number of moves, and the path of states. The path consists of the initial state and three steps, each showing a 3x3 grid of numbers.

```

print("Time taken: {:.6f} seconds".format(end_time - start_time))

```

Searching at depth: 0
Searching at depth: 1
Searching at depth: 2

Solution found!
Number of moves: 2

Step 0
(1, 2, 3)
(4, 0, 6)
(7, 5, 8)

Step 1
(1, 2, 3)
(4, 5, 6)
(7, 0, 8)

Step 2
(1, 2, 3)
(4, 5, 6)
(7, 8, 0)

Let's get you signed in
Upload files, use Voice without limits, and make Copilot yours with memory across conversations.
Sign in with Microsoft